

From Sold-Out to Sales Uplift: Causal Inference for Intelligent Inventory Management on Online Travel Platforms

Fanwei Zhu*
Hangzhou City University
Zhejiang, China
zhufanwei@zju.edu.cn

Zhuoran Zhuang*
Alibaba Group
Zhejiang, China
nantian@alibaba-inc.com

Detao Lv†
Alibaba Group
Zhejiang, China
detao.ltd@alibaba-inc.com

Manwei Li
Alibaba Group
Zhejiang, China
limanwei.lmw@alibaba-inc.com

Yao Yu
Alibaba Group
Zhejiang, China
sichen.yy@alibaba-inc.com

Abstract

Online Travel Platforms (OTPs) suffer significant revenue loss from “supply strikes”, where rooms with physical vacancies appear sold out due to delays in manual inventory updates from hotels. While proactively adding inventory is a potential solution, this intervention faces a dual risk: hotels may later reject the booking, and more critically, the intervention might not generate platform-wide revenue, but merely shift sales from a competing hotel. This paper is the first to formalize the inventory decision on OTPs as a causal inference problem. We propose CS²Net, a Causality-Driven, Scarcity- and Service-Aware Network that estimates the *platform-wide Individual Treatment Effect* of each inventory addition. CS²Net addresses the unique challenges of the OTP environment by integrating: (1) a Room Type Scarcity Representation module for inferring true room availability, (2) a Hotel Service-Engagement Representation module for predicting hotel acceptance, and (3) a bias-corrected causal framework to estimate platform-level uplift while mitigating selection bias. Extensive experiments and an online A/B test on a major OTP, demonstrate that CS²Net significantly increases confirmed bookings and platform revenue, generating over 10 million RMB in additional annual GMV. We also release the first causality dataset for third-party inventory management.

CCS Concepts

• Applied computing → Decision analysis.

Keywords

Sale shift, Causal Inference, Intelligent Inventory Management

ACM Reference Format:

Fanwei Zhu, Zhuoran Zhuang, Detao Lv, Manwei Li, and Yao Yu. 2026. From Sold-Out to Sales Uplift: Causal Inference for Intelligent Inventory Management on Online Travel Platforms. In *Proceedings of the ACM Web*

Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792791>

1 Introduction

The proliferation of third-party online travel platforms (OTPs) like Booking.com, Airbnb, and Fliggy have revolutionized the hospitality industry. Serving as intermediaries between hoteliers and customers, they offer convenient booking experiences by aggregating room availability from various hoteliers, driving significant changes in how accommodations are booked and managed.

A key operational challenge for OTPs is inventory management. For reasons of data privacy, security, and a desire to maintain direct control over room distribution, hoteliers typically do not grant OTPs direct access to their property management systems [3]. Instead, they provide only *partial inventory* to the platform, reserving the remainder for direct bookings, walk-in guests, or alternative channels. Inventory updates on the OTPs must therefore be performed manually by hoteliers.

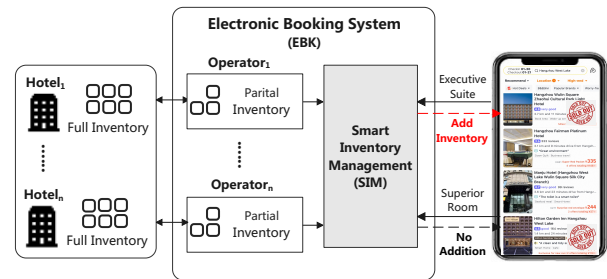


Figure 1: Intelligent Inventory Management on Fliggy.

This process introduces the risk of a *supply strike*—once the pre-allocated inventory is sold out on the platform, users can no longer book, even if the hotel still has vacant rooms. Supply strike leads to missed reservation opportunities and revenue loss for both the hotel and the platform. To mitigate these losses, it is important for OTPs to develop intelligent inventory management systems that can timely assist with inventory decisions for sold-out room types, determining whether to proactively add inventory unit for a sold-out room type?

*Equal contribution

†Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792791>

Example of Inventory Management System. Fig. 1 illustrates Fliggy’s intelligent inventory management process. Hoteliers provide partial inventory via the Electronic Booking System (EBK). When a room type sells out, the Smart Inventory Management (SIM) system evaluates signals such as predicted availability and hotel acceptance to decide whether to add inventory. If positive, the room is relisted; otherwise, it remains sold out.

Challenges. Unlike traditional inventory management that optimizes revenue for a *single hotel* [9, 39], OTPs should not only consider if adding inventory leads to a successful reservation for the target hotel, but more importantly, if it increases overall platform sales rather than merely shifting demand between competing hotels. Existing predictive models [22, 38] cannot answer this counterfactual question: *what is the platform-wide impact if we add inventory versus if we do not?*

Specifically, for intelligent inventory management on OTPs, we must address three unique challenges:

- *Inferring true inventory availability.* Without direct access to a hotel’s system, the OTP must estimate whether additional inventory exists for a sold-out room type.
- *Predicting hotel acceptance.* Even with availability, hotels may reject bookings for strategic reasons, harming user experience. The system must reliably predict the hotel’s acceptance probability.
- *Estimating platform-wide uplift.* As many substitute hotels exist on the platform, adding inventory for a sold-out room may not increase total bookings, but simply shifting demand away from competing hotels. The system must avoid such *sales shifts* and ensure genuine platform-level sale uplift.

Our Proposal. In this paper, we formalize the inventory addition decision on third-party platforms as a causal inference problem. Our goal is to estimate the platform-level treatment effect for each sold-out room type to ensure interventions create net-positive value. To achieve this, we propose CS²Net, a Causality-Driven, Scarcity- and Service-Aware Network. The architecture is specifically designed to address the core challenges of this problem: *Firstly*, to infer true inventory availability, a Room Type Scarcity Representation (RSR) module learns a room’s actual scarcity by analyzing both its internal demand patterns and the inventory of external competitors. *Secondly*, to predict whether a hotel will accept the booking, a Hotel Service-Engagement Representation (HSR) module models the hotel’s engagement or cooperation reliability in the OTP’s inventory management service using a dedicated hotel-room type graph enriched with historical reservation and rejection statistics. *Finally*, these rich representations are fed into a bias-corrected, multi-task causal framework that jointly predicts purchase and acceptance probabilities, enabling precise estimation of platform-level treatment effect.

Contributions. Our key contributions are threefold:

- We propose the first causal solution for third-party inventory management problem, aiming to estimate the net platform-wide impact of an intervention, explicitly modeling and mitigating sales shift effect- a critical challenge overlooked by traditional single-hotel revenue management.

- We propose CS²Net, a novel causal inference architecture that addresses the practical challenges of third-party platforms by introducing dedicated modules to infer true room availability and, crucially, to predict hotel acceptance behavior- a key but often ignored factor in the OTP booking process.
- We validate CS²Net through extensive offline experiments and a large-scale online A/B test, demonstrating significant business value with **over 10 million RMB in additional annual GMV**. We publicly release the code and the first causality dataset ¹ to facilitate further research on third-party inventory management.

2 Related Work

From the *problem perspective*, existing research on inventory management mainly focuses on revenue optimization for a *single party*, such as a hotel or an airline, using overbooking and pricing strategies. For example, *Harris et al.* [9] predicted user no-show probabilities to make overbooking decisions safely. *Zhu et al.* [39] proposed an elastic demand function for hotel occupancy prediction and derived optimal pricing strategies based on the estimated demand. In the airline industry, demand models have been proposed for dynamic seat allocation and pricing [1, 22]. These methods are powerful for a single business but are fundamentally ill-suited for a third-party OTP. Unlike prior work, our focus is on optimizing platform-wide uplift rather than single-hotel revenue. Beyond demand modeling, we also consider hotel service engagement to predict order acceptance, which is crucial for third-party platforms but overlooked in existing work.

From the *technique perspective*, traditional approaches often use handcrafted rules or optimization heuristics, which assume fixed demand patterns and lack adaptability to dynamic platform contexts. Recently, learning-based approaches have been introduced to handle the complexity of modern revenue management. For example, *Shihab et al.* [22] employed reinforcement learning to manage airline seat inventory under cancellations and no-shows. In parallel, causal inference has emerged as a powerful tool for decision-making across various domains like medicine and economics [27, 40]. For example, *Frauen et al.* [4] leveraged causal methods to assess treatment effects for low back pain, guiding better medical decisions. *Wang et al.* [27] used causal effect estimation to improve recommendation quality by modeling counterfactual user behavior. Despite its success in other domains, causal inference remains largely unexplored in inventory optimization on OTPs. Existing methods cannot be adapted to model the platform-wide causal effects of adding inventory for a sold-out room. Our work address this by explicitly modeling the platform-level ITE of an inventory addition, accounting for both the direct effect on the target hotel and potential sales shifts across its competitors.

3 Preliminaries and Problem

3.1 Preliminary Experiments on Sales Shifts

As motivated in Section 1, the decision to add inventory should aim to increase overall platform sales rather than merely shifting demand between competing hotels. To empirically validate the

¹Dataset and Code: <https://github.com/CS2NET/CS2Net>

phenomenon of undesired sales shifts and quantify the impact of inventory interventions, we conduct a randomized control trial (RCT) [26] involving 5,000 frequently sold-out hotels on Fliggy.

These hotels are randomly divided into two groups: 2,500 in the *treatment group* and 2,500 in the *control group*. For each sold-out room type in the treatment group, we add one additional inventory unit (i.e., an intervention), while no inventory is added in the control group. To assess possible sales shifts, we further identify a set of *similar hotels* for each target hotel based on location, room type, and star rating.

The experiment lasts two weeks. For each hotel and its similar counterparts, we compute the sequential growth rate of bookings (R_s) before and after the intervention, and further calculate the Difference-in-Differences (R_d) between treatment and control groups to isolate the causal effect of inventory addition.

From Table 1, we observe that inventory addition boosts bookings at the target hotels (+2.709% increase in R_d), validating that the intervention is effective for sold-out hotels. However, bookings at similar hotels experience a noticeable decline (−0.979% in R_d), indicating a significant *sales shift* effect. Furthermore, we find that 36.6% of the new bookings in the treatment group were later rejected by hoteliers, which contribute no actual revenue but can create poor user experiences.

Table 1: Sequential growth rates (R_s) and Difference-in-Differences (R_d) for target and similar hotels. $R_s = \frac{n_2 - n_1}{n_1} \times 100\%$, n_1 and n_2 are bookings before and after intervention.

Metric	Target Hotels	Similar Hotels
R_s^T (Treatment Group)	+3.041%	−0.541%
R_s^C (Control Group)	+0.332%	+0.438%
$R_d = R_s^T - R_s^C$	+2.709%	−0.979%

These findings demonstrates that *simplistic inventory addition is not practical*, highlighting the need of a principled approach that not only predicts **local outcomes** (e.g., bookings and acceptance) but also models **global impact** (e.g., platform-wide uplift) of an intervention. Basically, it requires answering a *counterfactual question*: For any sold-out room type, what would be the *platform-wide outcome* if we add inventory versus if we do not? Therefore, a causal framework that explicitly quantifies the treatment effect of inventory interventions is a natural and principled choice for our problem.

3.2 Problem Formulation

We formulate the inventory addition task using the potential outcomes framework [18] to estimate the causal impact of our decisions. Our goal is to determine the platform-wide effect of intervening on a sold-out room type x_i .

Let $D_i \in \{0, 1\}$ represent the treatment variable indicating whether an additional inventory unit is added for i ($D_i = 1$) or not ($D_i = 0$). The outcome of interest is a successful booking. To quantify the effects of the treatment, we define a model $F(x_k, D_i = d)$ which estimates the probability of a successful booking for a room type x_k under the condition that room x_i receives treatment d . Using this model, we estimate three levels of Individual Treatment Effects (ITEs):

(1) *Self-level ITE*. The direct causal effect of adding inventory to the sold-out room type x_i , defined as:

$$ITE_{\text{self}}(x_i) = F(x_i, D_i = 1) - F(x_i, D_i = 0) \quad (1)$$

(2) *Similar-level ITE*. Adding inventory to x_i can influence bookings for other similar room types $x_j \in \mathcal{S}_i$ (e.g., in nearby hotels or with comparable price/amenities). The indirect impact for all competing rooms is:

$$ITE_{\text{similar}}(x_i) = \sum_{x_j \in \mathcal{S}_i} [F(x_j, D_i = 1) - F(x_j, D_i = 0)] \quad (2)$$

(3) *Platform-level ITE*. To capture both direct and indirect effects, the platform-wide effect is defined as:

$$ITE_{\text{platform}}(x_i) = ITE_{\text{self}}(x_i) + ITE_{\text{similar}}(x_i) \quad (3)$$

PROBLEM 1 (INVENTORY ADDITION DECISION). *Given a sold-out room type x_i , our objective is to add inventory only when the estimated platform-level effect is positive:*

$$\text{Add inventory if } ITE_{\text{platform}}(x_i) \geq 0 \quad (4)$$

This rule ensures that the intervention contributes genuine platform-wide uplift rather than merely shifting sales across hotels.

As shown in Appendix A, a conservative lower bound of the platform-level ITE can be derived as:

$$\begin{aligned} ITE_{\text{platform}}(x_i) &\geq ITE'_{\text{platform}}(x_i) \\ &= F(x_i, D_i = 1) - \sum_{x_j \in \mathcal{S}_i} F(x_j, D_i = 0) \end{aligned} \quad (5)$$

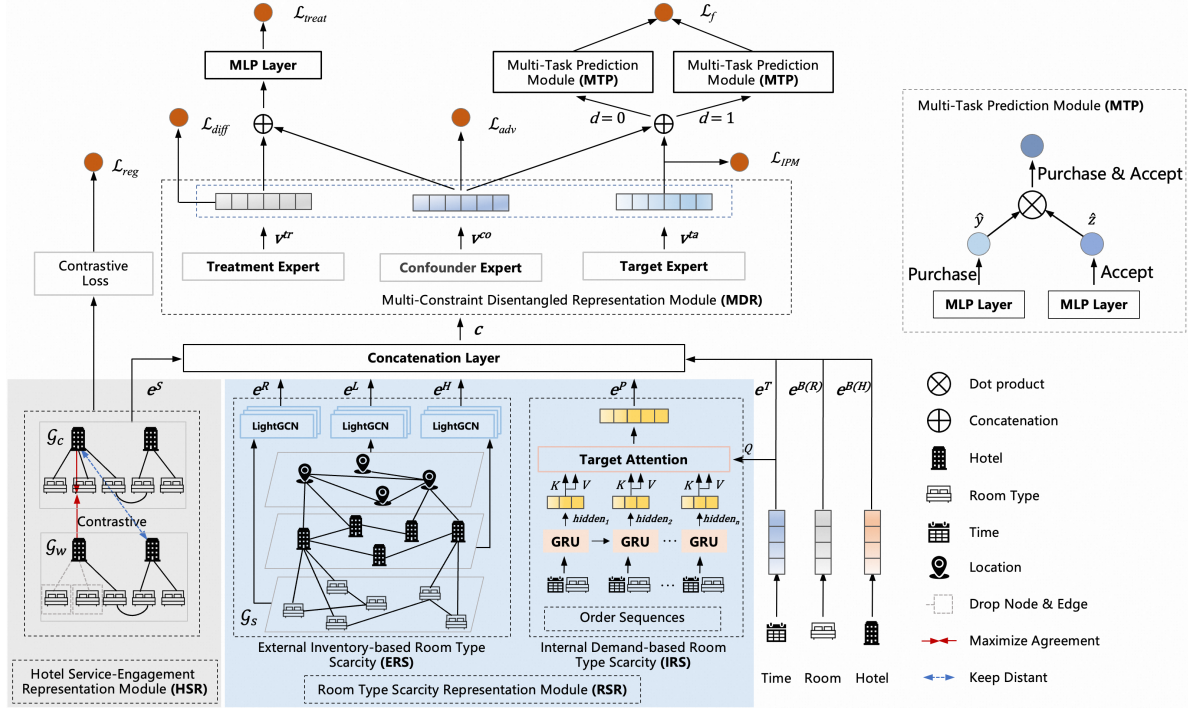
This formulation simplifies our decision problem into estimating success probability for the treated room and the baseline success probability of similar rooms. The next section presents our network architecture designed to accurately predict these probabilities.

4 CS²Net

4.1 Overview

The CS²Net is designed to implement our causal decision rule by estimating the success probability, $F(x, d)$. To achieve this, we decompose this probability into two components: (1) the probability that a user books the room, denoted by $\hat{y}(x, d)$, and (2) the probability that the hotel accepts the booking, denoted by $\hat{z}(x, d)$. Thus, our model estimates the overall success probability as: $F(x, d) = \hat{y}(x, d) \cdot \hat{z}(x, d)$.

As shown in Figure 2, our architecture is built to predict these two components through *representation learning* and *causal estimation*. The Room Type Scarcity Representation (RSR) module models inventory scarcity using both internal and external signals, serving as the primary input for predicting the purchase probability $\hat{y}$. The Hotel Service-Engagement Representation (HSR) module captures each hotel’s cooperation level in the platform’s inventory management service, providing the key input for predicting the acceptance probability $\hat{z}$. These representations, along with contextual features (e.g., hotel, room, and temporal attributes), are then processed by the causal estimation framework, which first uses a Multi-Constraint Disentangled Representation (MDR) module to mitigate selection bias, and then jointly predicts $\hat{y}$ and $\hat{z}$ with a multi-task Prediction (MTP) module to compute the platform-level ITE and make a final decision.

Figure 2: The Architecture of CS²Net.

4.2 Room Type Scarcity Representation Module

The Room Type Scarcity Representation (RSR) Module is designed to better predict the actual inventory status of room types, which learns scarcity representations from two perspectives: external inventory-based scarcity and internal demand-based scarcity.

4.2.1 Internal Demand-based Room Type Scarcity (IRS). The inventory status of room-type is inherently correlated with its demand—higher demand typically leads to greater scarcity. Therefore, this model focuses on capturing the scarcity of room types from the perspective of internal demand. By leveraging recent sales data and time-series modeling, IRS predicts the demand for a given room type on specific dates.

Specifically, the IRS module uses the past 30 days of sales data (i.e., number of room-night bookings) and time-related features as inputs, and employ Gated Recurrent Units (GRU) [2, 5] to obtain the sequential demand representations. For each time step j , the GRU takes $t_{1:j}$ (time steps up to day j) and $s_{1:j}$ (sales up to j) and output a hidden state $hidden_j$:

$$hidden_j = GRU(t_{1:j}, s_{1:j}) \quad (6)$$

Given the relatively simply nature of the input data and task, GRU is a suitable and efficient choice that aligns with our objective. To focus on the most relevant time steps for predicting the demand of time t_i , we further apply an attention mechanism to compute the weight for each j w.r.t t_i :

$$att_{i,j} = \mathcal{H}(\text{concat} \langle query_i, hidden_j \rangle) \quad (7)$$

where $query_i = e_i^T$ is the basic feature representation of time t_i and $\mathcal{H}$ is a MultiLayer Perceptron (MLP) network.

The final internal demand representation e_i^P is computed as a weighted sum of all hidden states:

$$e_i^P = \sum_{j=1}^{30} \frac{\exp(att_{i,j})}{\sum_{k=1}^{30} \exp(att_{i,k})} \cdot hidden_j \quad (8)$$

4.2.2 External Inventory-based Room Type Scarcity (ERS). From an external perspective, room-type scarcity is often correlated across similar hotels. Since users typically consider multiple substitutable hotels within the same area, low inventory in one hotel can signal similar scarcity in others. Moreover, the inventory status of other room types within the same hotel or nearby hotels may also reflect the availability of a specific room type.

To capture these interdependencies, we construct a heterogeneous graph $\mathcal{G}_s$, the Location–Hotel–Room Type Inventory Status Graph, as illustrated in Figure 3. This graph comprises three types of nodes: location nodes l_i , hotel nodes h_i , and room-type nodes r_i . Locations are defined using a fine-grained GeoHash-based partitioning scheme [23, 29], where each hotel is mapped to a geographic cell based on its latitude and longitude, allowing nearby hotels to share a location node. Room types are categorized by functional characteristics such as bed type (e.g., single or double). Each room-type node is associated with attributes reflecting its inventory status (e.g., available, sold out). The graph includes five types of relations to encode inventory-related dependencies: (1) *Location adjacency* that connects geographically adjacent locations, (2) *Hotel similarity* that links hotels with similar attributes, (3) *Location–hotel inclusion* that associates each hotel with its location, (4) *Hotel–room type inclusion* that connects hotels to their listed room types, and (5)

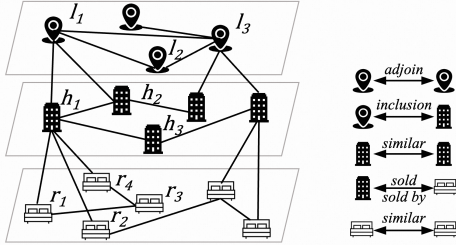


Figure 3: Location-Hotel-Room Type Inventory Status Graph.

Room-type similarity that links functionally similar room types (e.g., size, amenities, or pricing level). The intuition is to propagate inventory signals through both hotel-level and room-level similarity structures to infer scarcity more accurately. We apply LightGCN [11], a lightweight graph convolutional framework, to learn node embeddings from this graph. Specifically, for each node i , the information propagation at k^{th} layer follows:

$$e_i^{(k)} = \sum_{j \in N_i} \frac{1}{\sqrt{N_i} \sqrt{N_j}} e_j^{(k-1)}, \quad (9)$$

where $e_i^{(k)}$ and $e_i^{(k-1)}$ are the representations of node i and node j on the k^{th} and $(k-1)^{th}$ layers, respectively. The embedding $e_i^{(0)}$ is randomly initialized at the beginning of the training.

By stacking K layers of message passing and aggregation, each room-type node integrates information from multi-hop neighbors. The final node representation is obtained by aggregating the representations across all layers:

$$e_i = \sum_{k=0}^K e_i^{(k)}. \quad (10)$$

where e_i denotes the final representation of node i . For clarity, we denote the final representations of room type r_i , hotel h_i , and location l_i in the graph $\mathcal{G}_s$ as e_i^R , e_i^H , and e_i^L , respectively. These representations are combined with the internal demand-based scarcity representation from the IRS component to form a comprehensive scarcity representation for the RSR module.

4.3 Hotel Service-Engagement Representation Module

As hotels may reserve some rooms for direct or walk-in bookings, or allocate them to alternative reservation channels, they may reject platform-added inventory even when they have physical vacancy. The Hotel Service Representation (HSR) Module is designed to capture a hotel's engagement in the OTP's inventory management service, which is critical in predicting whether newly-added inventory will be accepted by the hotel.

Hotel-Room Type-Service Graph. To capture informative signals, we construct a heterogeneous Hotel-Room Type-Service Graph $\mathcal{G}_c$ (see Figure 4) to model two key relationships: *Hotel-room type inclusion* that connects a hotel node to the room type nodes it offers, and *Room-type similarity* that connects similar room types within the hotel and across similar hotels nearby. Each room-type node in $\mathcal{G}_c$ carries engagement-related statistics, such as annual order quantity and rejection rate. This structure allows the HSR module to incorporate both a hotel's own behavior and patterns

from similar competitors, producing a more robust representation of service engagement.

However, learning such representations faces two challenges: (1) Hotel service engagement is a latent attribute that cannot be explicitly labeled. Lack of explicit labels makes supervised learning infeasible. (2) Room types with no orders always have a rejection rate of zero- not because the hotel never rejects, but due to the lack of orders. These zero-order room types provide ambiguous signal that must be handled.

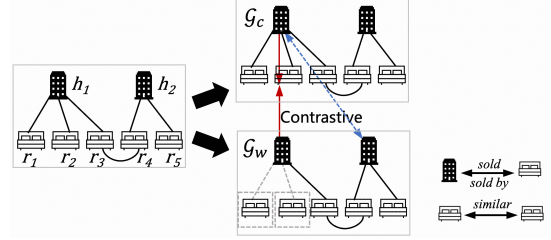


Figure 4: Hotel-Room Type-Service Graph.

Graph Contrastive Learning with Data Augmentation. To tackle these challenges, we integrate Graph Contrastive Learning [13, 24, 28, 32] with a structure-based augmentation [20, 35] strategy. Specifically, we construct two graph views: the original $\mathcal{G}_c$ and an augmented subgraph $\mathcal{G}_w$ where room-type nodes with zero orders and their connected edges are removed to eliminate noise. Both graphs are processed using LightGCN [11] to generate node embeddings. In particular, for each node i , we obtain two representations e_i^c from $\mathcal{G}_c$ and e_i^w from $\mathcal{G}_w$, forming positive sample pairs. Negative pairs are constructed by randomly sampling representations from $\mathcal{G}_w$ that do not correspond to node i . We then optimize the widely used InfoNCE loss [8] to enforce consistency between the representations of the same node across the two views while differentiating representations across nodes. The loss functions for hotels $\mathcal{L}_H$ and room types $\mathcal{L}_R$ are defined as:

$$\begin{aligned} \mathcal{L}_H &= \frac{1}{|H|} \sum_{h \in H} -\log \frac{\exp(s(e_h^c, e_h^w)/\tau)}{\sum_{v \in H} \exp(s(e_h^c, e_v^w)/\tau)}, \\ \mathcal{L}_R &= \frac{1}{|R|} \sum_{r \in R} -\log \frac{\exp(s(e_r^c, e_r^w)/\tau)}{\sum_{b \in R} \exp(s(e_r^c, e_b^w)/\tau)}, \end{aligned} \quad (11)$$

where $s(\cdot)$ is the cosine similarity function and τ is a hyper-parameter called *temperature*. The final contrastive loss $\mathcal{L}_{reg}$ is the sum of the two losses:

$$\mathcal{L}_{reg} = \mathcal{L}_H + \mathcal{L}_R. \quad (12)$$

By minimizing $\mathcal{L}_{reg}$, HSR learns the hotel service-engagement representation e_i^S , which is later integrated into the causal inference framework to predict the impact of inventory addition.

4.4 Multi-Constraint Disentangled Representation Module

Through the above sub-modules, CS²Net constructs a comprehensive feature representation for each room type by integrating multiple factors. Specifically, the representations of hotel service e_i^S , external and internal scarcity e_i^R , e_i^H , e_i^L , e_i^P , together with the basic information representations of the room type $e_i^{B(R)}$, the hotel

$e_i^{B(H)}$ and the time factor e_i^T to form the covariate c_i . The causal estimation framework then leverages c_i to estimate the ITE for each sold-out room type.

To mitigate selection bias, we introduce the Multi-Constraint Disentangled Representation Module (MDR), which disentangles different causal factors through expert networks and imposes constraints to ensure effective separation of representations.

Disentangled Representations. Inspired by disentangled representation learning and adversarial training, MDR decomposes c_i into three distinct expert-learned representations:

- Treatment-specific representation v_i^{tr} , which captures factors that influence treatment assignment (whether inventory is added) but do not directly impact the outcome.
- Confounder representation v_i^{co} , which encodes factors affecting both treatment assignment and outcome.
- Target-specific representation v_i^{ta} , which encodes factors that directly determine the outcome (successful booking) but are independent of treatment assignment.

These representations are learned using expert networks, implemented as MLPs.

Multi-Constraint Learning. While traditional methods like DR-CFR [10] use Integral Probability Metric (IPM) [14, 30] to constrain the learning, they do not explicitly enforce independence between these representations, leading to suboptimal performance. To address this, we introduce two additional loss functions: an adversarial loss and a disentangled loss, in addition to IPM.

First, to mitigate selection bias, we employ adversarial learning [16] to ensure that the distribution of the confounder representation v_i^{co} is consistent between the treatment and control groups. The key idea is to make it difficult for a discriminator to predict whether a sample belongs to the treatment or control group based on v_i^{co} . The adversarial loss is defined as:

$$\mathcal{L}_{adv} = \min_{\theta_{EC}} \left(\lambda \max_{\theta_D} \left(\sum_{i=1}^n d_i \log(J(E_C(c_i))) \right) \right) \quad (13)$$

where the discriminator $J(\cdot)$ is trained to predict the treatment d_i from v_i^{co} using an MLP and E_C denotes the confounder expert network.

Second, to ensure that capture distinct aspects of the data, we introduce a disentanglement constraint that maximizes the differences between them. The disentangled loss is defined as:

$$\mathcal{L}_{diff} = \sum_{i=1}^n \|(v_i^{tr})^T v_i^{co}\| + \|(v_i^{co})^T v_i^{ta}\| + \|(v_i^{tr})^T v_i^{ta}\| \quad (14)$$

Third, to ensure that v^{ta} is independent of d due to the collider structure [10, 33], we introduce a balancing regularization term $\mathcal{L}_{IPM}$, following the approach of DR-CFR. Specifically, we enforce a balanced distribution of v^{ta} between the treatment and control groups using IPM. The IPM loss is defined as:

$$\mathcal{L}_{IPM} = IPM(v^{ta} \cdot D, v^{ta} \cdot (1 - D)) \quad (15)$$

where D denotes the treatment vector, $v^{ta} \cdot D, v^{ta} \cdot (1 - D)$ represent the target-specific representations for the treatment and control groups, respectively.

4.5 Multi-Task Prediction Module

To accurately estimate the probability of a successful transaction, it is essential to consider both the likelihood of the added inventory being purchased $\hat{y}$ and the likelihood of the hotel accepting the order $\hat{z}$. Since the hotel's acceptance decision occurs after a user makes a purchase, we adopt the Entire Space Multi-Task Model (ESMM) framework [17], which leverages the sequential dependence between purchase and acceptance to model these two tasks.

Prediction of Purchase and Acceptance Probabilities. The MTP module takes the target-specific representation v_i^{ta} and the confounder v_i^{co} representation as inputs, which are concatenated to form a combined representation:

$$v_i^{ao} = \text{concat}(v_i^{ta}, v_i^{co}) \quad (16)$$

Two separate MLP networks are then used to predict the purchase probability $\hat{y}_i$, parameterized by θ_y and the acceptance probability $\hat{z}_i$, parameterized by θ_z . For the treatment group $d_i = 1$ (i.e., inventory is added), the predictions are formulated as:

$$\hat{y}_i = f_t(v_i^{ao}; \theta_y) \quad \hat{z}_i = f_t(v_i^{ao}; \theta_z) \quad (17)$$

For the control group $d_i = 0$, the same module structure is used but with a separate set of neural network parameters.

Multi-tasking Learning Objective. The MTP model simultaneously optimizes two objectives: (1) the purchase probability $\hat{y}_i$ and (2) the joint probability of purchase and acceptance $\hat{y}_i \hat{z}_i$. The overall loss function is thus defined as:

$$\begin{aligned} \mathcal{L}_f = & \sum_{i \in \mathcal{T}} l(y_i, f_t(v_i^{ao}; \theta_y)) + \sum_{i \in \mathcal{C}} l(y_i, f_c(v_i^{ao}; \theta_y)) \\ & + \sum_{i \in \mathcal{T}} l(y_i z_i, f_t(v_i^{ao}; \theta_y) f_t(v_i^{ao}; \theta_z)) \\ & + \sum_{i \in \mathcal{C}} l(y_i z_i, f_c(v_i^{ao}; \theta_y) f_c(v_i^{ao}; \theta_z)) \end{aligned} \quad (18)$$

where $\mathcal{T}$ and $\mathcal{C}$ represent the treatment and control groups, respectively, $l(\cdot)$ is the cross-entropy loss function.

Since it is impossible to observe the factual outcomes for the control group when no inventory is added, the factual outcomes are constructed from successful transactions of similar room types $\mathcal{S}_i$ that share attributes such as star ratings, prices, and locations. Therefore, $\sum_{i \in \mathcal{C}} f_c(v_i^{ao}; \theta_y) f_c(v_i^{ao}; \theta_z)$ in Eq. 18 corresponds precisely to the term $\sum_{x_j \in \mathcal{S}_i} F(x_j, d_i = 0)$ in Eq. 23, enabling the calculation of $ITE'_{platform}$ defined in Section 3.

4.6 Training

During the model training phase, the overall loss function of CS²Net consists of six components, formulated as:

$$\mathcal{L}_M = \mathcal{L}_f + \mathcal{L}_{treat} + \alpha_1 \mathcal{L}_{adv} + \alpha_2 \mathcal{L}_{diff} + \alpha_3 \mathcal{L}_{IPM} + \alpha_4 \mathcal{L}_{reg}. \quad (19)$$

where $\alpha_1, \alpha_2, \alpha_3$, and α_4 are hyperparameters that control the trade-off between multiple loss terms. $\mathcal{L}_f$ is the multi-task factual loss (Eq. 18), $\mathcal{L}_{adv}$ is the adversarial loss (Eq. 13), $\mathcal{L}_{diff}$ is the disentanglement loss (Eq. 14), $\mathcal{L}_{IPM}$ is the balancing regularization (Eq. 15), and $\mathcal{L}_{reg}$ is the contrastive learning loss (Eq. 12). Additionally, CS²Net incorporates a cross-entropy loss $\mathcal{L}_{treat}$ for explicitly predicting the treatment assignment across n samples, following the approach in [34]:

$$\mathcal{L}_{treat} = -\frac{1}{n} \sum_{i=1}^n (d_i \log(MLP(v_i^{ro})) + (1 - d_i) \log(1 - MLP(v_i^{ro}))) \quad (20)$$

where $v_i^{ro} = \text{concat}(v_i^{tr}, v_i^{co})$.

5 Experiments

5.1 Experimental Settings

Table 2: Statistics of Datasets.

Category	Training	Validation	Testing
Room Types	482,612	144,326	163,570
Hotels	93,972	38,140	48,884
Total Samples	7,744,275	806,593	918,620
Treatment Samples	4,495,830	468,170	534,109
Control Samples	3,248,446	338,423	384,511

5.1.1 Dataset Description. We introduce the first causal dataset for OTP inventory management, constructed with operational data from Fliggy. The dataset spans 120,426 hotels across 300 cities and is chronologically split into training, validation, and testing sets. The statistics of the dataset are shown in Table 2, where *treatment samples* refer to historical sold-out cases where additional inventory was proactively added, while *control samples* are those where no inventory addition was made. The ground-truth labels are derived directly from the transaction logs, which contain the order ID, hotel ID, booking status, hotel acceptance status, etc. For example, if adding inventory for room-type i results in a booking for it and hotel accepts this booking, the label is ($y_i = 1, z_i = 1$).

5.1.2 Baselines. To validate the effectiveness of CS²Net, we compare it against several state-of-the-art causal effect estimation models. The selected baselines represent diverse strategies for mitigating selection bias and improving ITE estimation, including several *representation constraint methods*: TARNet [19], Dragonnet [21], DR-CFR [10], EFIN [15], an *adversarial learning method* GANITE [31], and two full-space modeling method DESCN [37], ECUP [12]. Details of the baselines are provided in Appendix B.

5.1.3 Metrics. We adopt Area Under the Uplift Curve (AUUC) [7, 36], a standard metric in causal effect estimation, to measure the overall uplift performance of different models in offline experiments. AUUC assesses a model’s ability to effectively rank cases by their predicted treatment effect, prioritizing interventions for those that will generate the most incremental successful outcomes compared to taking no action. A higher AUUC indicates superior overall uplift performance. In addition, we evaluate the model’s ranking quality specifically within the treatment group using two standard metrics. HitRate@ k measures the proportion of top $k\%$ predictions where a positive outcome (successful booking) occurs and Precision@ k computes the proportion of positive outcome among the top $k\%$ inventory additions. Formal definitions for all metrics are provided in Appendix C.

5.1.4 Parameters Settings. We evaluate the impact of hyperparameters $\alpha_1, \alpha_2, \alpha_3$, and α_4 in Eq. 19 on the performance of our model. As summarized in Table 6, the model achieves the best performance with the following settings: $\alpha_1 = 10^{-5}$, $\alpha_2 = 10^{-1}$, $\alpha_3 = 10^{-2}$, and $\alpha_4 = 10^{-4}$, which are used in subsequent experiments. More details can be found in Appendix E.

5.2 Offline Experiments

5.2.1 Comparison with Baselines. The experimental results in Table 3 demonstrate that CS²Net achieves the best performance, significantly outperforming all baselines across all evaluation metrics with a statistical significance of $p < 0.01$. This validates the effectiveness of CS²Net in making optimal inventory addition decisions for sold-out room types. Specifically, CS²Net achieves an AUUC of 0.8587, exceeding the best-performing baseline ECUP by 3.38%. HitRate@10% of CS²Net reaches 27.39%, outperforming ECUP by 2.8% and Precision@10% improves to 10.67%, exceeding ECUP by 1.34%. This improvement can be attributed to CS²Net’s well-designed architecture that enforces independence among v^{tr} , v^{co} and v^{ta} via disentanglement loss, aligns confounder distributions across treatment groups with adversarial learning, and leverages RSR and HSR to extract effective scarcity and service representations. We also compare the training and inference time for different methods, and the computational cost of CS²Net is comparable to that of baselines. See Appendix D for more details.

Table 3: Comparison of Different Methods.

Method	AUUC↑	HitRate@ $k\uparrow$		Precision@ $k\uparrow$	
		$k=10$	$k=20$	$k=10$	$k=20$
TARNet	0.799	19.85%	31.61%	8.08%	5.73%
GANITE	0.797	19.70%	31.19%	8.06%	5.48%
Dragonnet	0.812	23.03%	33.26%	8.63%	6.17%
DR-CFR	0.804	21.97%	32.64%	8.28%	6.10%
DESCN	0.800	21.58%	32.28%	8.22%	6.01%
EFIN	0.817	23.76%	33.67%	9.01%	6.62%
ECUP	0.825	24.59%	34.62%	9.33%	6.70%
CS²Net	0.859	27.39%	39.26%	10.67%	7.76%

5.2.2 Ablation Study. To further investigate the contribution of key submodules in CS²Net we conduct an ablation study by comparing it against five variant models, each with a specific component removed: (1) *C-Adv*: Removes the adversarial loss in the Multi-Constraint Disentangled Representation (MDR) module; (2) *C-Dis*: Removes the disentanglement loss in MDR; (3) *C-ERS*: Removes the External Inventory-based Scarcity module in Room Type Scarcity Representation (RSR); (4) *C-IRS*: Removes the Internal Demand-based Scarcity module in RSR; (5) *C-HSR*: Removes the Hotel Service Representation (HSR) module.

The ablation results in Table 4 validate that each component in CS²Net plays a critical role in optimizing inventory decisions and ensuring platform-wide revenue growth. Specifically, we observe that (1) CS²Net outperforms *C-ERS*, *C-IRS*, and *C-HSR* by at least 1.15% (AUUC), 1.28% (HitRate@10%), and 0.51% (Precision@10%), confirming that RSR and HSR effectively capture room type scarcity and hotel service quality. (2) CS²Net outperforms *C-Adv* and *C-Dis*

across all metrics, demonstrating the effectiveness of adversarial and disentanglement losses in MDR.

Table 4: Ablation Study of CS²Net.

Variant	AUUC↑	HitRate@k↑		Precision@k↑	
		k=10	k=20	k=10	k=20
C-Adv	0.834	25.09%	36.34%	9.51%	6.44%
C-Dis	0.833	25.07%	36.28%	9.49%	6.41%
C-ERS	0.847	26.11%	38.25%	10.16%	7.06%
C-IRS	0.841	25.55%	37.60%	9.89%	6.79%
C-HSR	0.844	25.80%	37.85%	10.03%	6.92%
CS²Net	0.859	27.39%	39.26%	10.67%	7.76%

5.3 System Deployment & Online Test

With the success in offline experiments, we deploy CS²Net in the live environment of Fliggy and evaluate its real-world effectiveness.

System Deployment. The deployment of CS²Net involves both offline training and online serving, as shown in Fig. 5. For offline training, we first store and preprocess collected transaction behaviors, hotel interactions, and room-type inventory changes in our self-developed MaxCompute Platform (MCP) as structured log records. The processed data is used to generate training and test datasets. Distributed training is conducted on Alibaba's Platform of Artificial Intelligence (PAI), after which the trained CS²Net model is deployed on a Real-Time Prediction (RTP) platform for online inference.

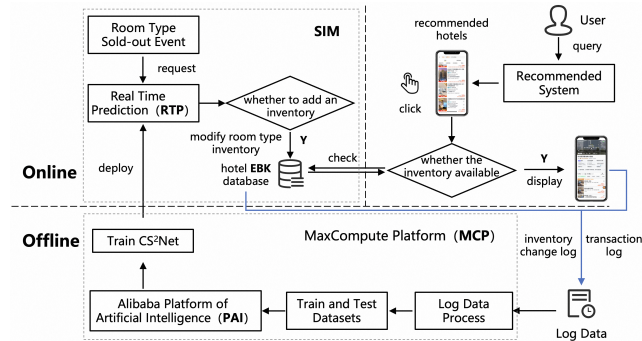


Figure 5: Offline Training and Online Deployment.

During online serving, when a sold-out event occurs, contextual features (e.g., time, location, and inventory status of similar rooms) are sent to RTP, which invokes CS²Net to estimate the ITE of adding inventory. If the decision is positive, the Electronic Booking System (EBK) updates the inventory from 0 to 1, relisting the room type. Meanwhile, when users search and click a hotel in the app, a real-time inventory check ensures that only available room types are displayed, maintaining a seamless booking experience.

Online Test. A two-week online test was conducted from July 3 to July 16, 2025, using the strongest offline baseline ECUP as comparison model. We evaluate platform-wide uplift using two

business-oriented metrics *Order* (i.e., the total number of confirmed bookings) and *Acceptance Rate* (i.e., the proportion of added inventory being accepted by hoteliers). We randomly assign hotels into two groups: Base group using ECUP and Test group using CS²Net for inventory interventions. To ensure a fair comparison, both groups run the baseline model during the first 7 days to validate that the hotel allocation was unbiased. From day 8 onward, the Test group switches to CS²Net for inventory decisions, while the Base group continues with ECUP.

As shown in Figure 6, switching the Test group to CS²Net resulted a 1.85% increase in confirmed bookings and a 2.87% improvement in the hotel acceptance rate compared to the Baseline group. This performance translates to over 10 million RMB in additional annual GMV, demonstrating substantial business impact.

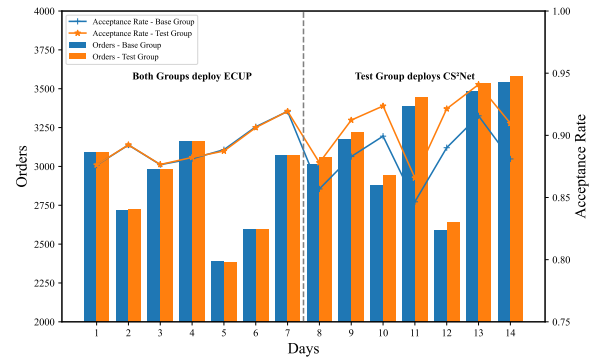


Figure 6: Online Orders and Acceptance Rate Comparison.

We further conduct a randomized controlled trials and confirm that CS²Net effectively grows the overall platform revenue with minimal sales displacement in real setting (details in Appendix G).

6 Conclusion and Future Work

In this work, we propose CS²Net, a causality-driven framework for intelligent inventory addition. By modeling real-time room scarcity and hotel service quality, CS²Net uses a causal inference model to accurately estimate the net platform-wide impact of each potential inventory addition, effectively addressing the sale shifts issue on the OTP platform and achieving significant revenue growth.

For future work, we plan to advance our framework in two directions: (1) enhancing demand prediction by incorporating richer dynamic signals (e.g., local events and weather) and more expressive sequential models to better capture market dynamics; and (2) extending our causal framework beyond binary treatments to model dosage effects (e.g., the optimal number of rooms to add) and to support other platform interventions such as dynamic pricing, enabling a more general decision-support engine for OTPs.

Acknowledgments

This work is supported by the Zhejiang Provincial Natural Science Foundation (No. LY24F020013), China. The authors would like to acknowledge the Supercomputing Center of Hangzhou City University and Zhejiang Provincial Engineering Research Center for Real-Time SmartTech in Urban Security Governance, for their support of the advanced computing resources.

References

- [1] Kavitha Balaiyan, RK Amit, Atul Kumar Malik, Xiaodong Luo, and Amit Agarwal. 2019. Joint forecasting for airline pricing and revenue management. *Journal of Revenue and Pricing Management* 18 (2019), 465–482.
- [2] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078* (2014).
- [3] Yufeng Dong and Liuyi Ling. 2015. Hotel Overbooking and Cooperation with Third-Party Websites. *Sustainability* 7, 9 (Aug. 2015), 11696–11712.
- [4] Dennis Frauen, Tobias Hatt, Valentyn Melnychuk, and Stefan Feuerriegel. 2023. Estimating average causal effects from patient trajectories. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 7586–7594.
- [5] Lingyue Fu, Jianghao Lin, Weiwen Liu, Ruiming Tang, Weinan Zhang, Rui Zhang, and Yong Yu. 2023. An F-shape Click Model for Information Retrieval on Multi-block Mobile Pages. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 1057–1065.
- [6] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 249–256.
- [7] Pierre Gutierrez and Jean-Yves G  rardy. 2017. Causal inference and uplift modelling: A review of the literature. In *International conference on predictive applications and APIs*. PMLR, 1–13.
- [8] Michael Gutmann and Aapo Hyv  rinen. 2010. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 297–304.
- [9] Shannon L Harris and Michele Samorani. 2021. On selecting a probabilistic classifier for appointment no-show prediction. *Decision Support Systems* 142 (2021), 113472.
- [10] Negar Hassanpour and Russell Greiner. 2019. Learning disentangled representations for counterfactual regression. In *International Conference on Learning Representations*.
- [11] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.
- [12] Yinqiu Huang, Shuli Wang, Min Gao, Xue Wei, Changhao Li, Chuan Luo, Yinhua Zhu, Xiong Xiao, and Yi Luo. 2024. Entire chain uplift modeling with context-enhanced learning for intelligent marketing. In *Companion Proceedings of the ACM Web Conference 2024*. 226–234.
- [13] Yangqin Jiang, Chao Huang, and Lianghao Huang. 2023. Adaptive graph contrastive learning for recommendation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4252–4261.
- [14] Fredrik Johansson, Uri Shalit, and David Sontag. 2016. Learning representations for counterfactual inference. In *International conference on machine learning*. PMLR, 3020–3029.
- [15] Dugang Liu, Xing Tang, Han Gao, Fuyuan Lyu, and Xiuqiang He. 2023. Explicit Feature Interaction-aware Uplift Network for Online Marketing. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4507–4515.
- [16] Pengfei Liu, Xipeng Qiu, and Xuanjing Huang. 2017. Adversarial multi-task learning for text classification. *arXiv preprint arXiv:1704.05742* (2017).
- [17] Xiao Ma, Liqin Zhao, Guan Huang, Zhi Wang, Zelin Hu, Xiaoqiang Zhu, and Kun Gai. 2018. Entire space multi-task model: An effective approach for estimating post-click conversion rate. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 1137–1140.
- [18] Donald B Rubin. 2005. Causal inference using potential outcomes: Design, modeling, decisions. *J. Amer. Statist. Assoc.* 100, 469 (2005), 322–331.
- [19] Uri Shalit, Fredrik D Johansson, and David Sontag. 2017. Estimating individual treatment effect: generalization bounds and algorithms. In *International conference on machine learning*. PMLR, 3076–3085.
- [20] Xiao Shen, Dewang Sun, Shirui Pan, Xi Zhou, and Laurence T Yang. 2023. Neighbor contrastive learning on learnable graph augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 9782–9791.
- [21] Claudia Shi, David Blei, and Victor Veitch. 2019. Adapting neural networks for the estimation of treatment effects. *Advances in neural information processing systems* 32 (2019).
- [22] Syed Arbab Mohd Shihab, Caleb Logemann, Deepak-George Thomas, and Peng Wei. 2019. Autonomous airline revenue management: A deep reinforcement learning approach to seat inventory control and overbooking. *arXiv preprint arXiv:1902.06824* (2019).
- [23] Ali Shiri, Asad YarAhmadi, Samira Keivanpour, and Amina Lamghari. 2024. Real-time RL-based Matching with H3 Geohash Partitioning in Smart Freight Platform. In *2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall)*. IEEE, 1–7.
- [24] Jie Shuai, Kun Zhang, Le Wu, Peijie Sun, Richang Hong, Meng Wang, and Yong Li. 2022. A review-aware graph contrastive learning framework for recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1283–1293.
- [25] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of Machine Learning Research* 9, Nov (2008), 2579–2605.
- [26] Haotian Wang, Wenjing Yang, Longqi Yang, Anpeng Wu, Liyang Xu, Jing Ren, Fei Wu, and Kun Kuang. 2022. Estimating Individualized Causal Effect with Confounded Instruments. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1857–1867.
- [27] Zhenlei Wang, Xu Chen, Rui Zhou, Quanyu Dai, Zhenhua Dong, and Ji-Rong Wen. 2023. Sequential recommendation with causal behavior discovery. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 28–40.
- [28] Jiancan Wu, Xiang Wang, Fuli Feng, Xiangnan He, Liang Chen, Jianxun Lian, and Xing Xie. 2021. Self-supervised graph learning for recommendation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 726–735.
- [29] Jia Xu, Fei Xiong, Zulong Chen, Mingyuan Tao, Liangyue Li, and Quan Lu. 2022. G2NET: A general geography-aware representation network for hotel search ranking. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4237–4247.
- [30] Liuyi Yao, Yaliang Li, Sheng Li, Mengdi Huai, Jing Gao, and Aidong Zhang. 2021. SCI: subspace learning based counterfactual inference for individual treatment effect estimation. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 3583–3587.
- [31] Jinsung Yoon, James Jordon, and Mihaela Van Der Schaar. 2018. GANITE: Estimation of individualized treatment effects using generative adversarial nets. In *International conference on learning representations*.
- [32] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.
- [33] Zhongqi Yue, Hanwang Zhang, Qianru Sun, and Xian-Sheng Hua. 2020. Inter-ventional few-shot learning. *Advances in neural information processing systems* 33 (2020), 2734–2746.
- [34] Guozhen Zhang, Jinwei Zeng, Zhengyue Zhao, Depeng Jin, and Yong Li. 2022. A Counterfactual modeling framework for churn prediction. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 1424–1432.
- [35] Jiaxing Zhang, Dongsheng Luo, and Hua Wei. 2023. Mixupexplainer: Generalizing explanations for graph neural networks with data augmentation. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3286–3296.
- [36] Yan Zhao, Xiao Fang, and David Simchi-Levi. 2017. Uplift modeling with multiple treatments and general response types. In *Proceedings of the 2017 SIAM International Conference on Data Mining*. SIAM, 588–596.
- [37] Kailiang Zhong, Fengtong Xiao, Yan Ren, Yaorong Liang, Wenqing Yao, Xiaofeng Yang, and Ling Cen. 2022. DESCN: Deep Entire Space Cross Networks for Individual Treatment Effect Estimation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4612–4620.
- [38] Fanwei Zhu, Wendong Xiao, Yao Yu, Zemin Liu, Zulong Chen, and Weibin Cai. 2024. Dynamic Hotel Pricing at Online Travel Platforms: A Popularity and Competitiveness Aware Demand Learning Approach. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Barcelona, Spain) (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 4641–4651. <https://doi.org/10.1145/3637528.3671921>
- [39] Fanwei Zhu, Wendong Xiao, Yao Yu, Ziyi Wang, Zulong Chen, Quan Lu, Zemin Liu, Minghui Wu, and Shenghua Ni. 2022. Modeling Price Elasticity for Occupancy Prediction in Hotel Dynamic Pricing. In *CIKM*. 4742–4746.
- [40] Feng Zhu, Mingjie Zhong, Xinxing Yang, Longfei Li, Lu Yu, Tiehua Zhang, Jun Zhou, Chaochao Chen, Fei Wu, Guanfang Liu, et al. 2023. DCMT: A Direct Entire-Space Causal Multi-Task Framework for Post-Click Conversion Estimation. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 3113–3125.

Appendix

A Derivation of Simplified Platform-Level ITE

We derive a simplified expression of the platform-level ITE (Eq. 3) used in the inventory addition decision.

For a sold-out room type x_i , the treatment $d_i \in \{0, 1\}$ indicates whether an additional inventory unit is added ($d_i = 1$) or not ($d_i = 0$).

The platform-level ITE considers both the treated room type and its substitutes S_i :

$$ITE_{\text{platform}}(x_i) = ITE_{\text{self}}(x_i) + ITE_{\text{similar}}(x_i) \quad (21)$$

$$= \left(F(x_i, D_i = 1) + \sum_{x_j \in S_i} F(x_j, D_i = 1) \right) - \left(F(x_i, D_i = 0) + \sum_{x_j \in S_i} F(x_j, D_i = 0) \right) \quad (22)$$

Since $F(x_i, D_i = 0) = 0$ (i.e., no booking can occur for a sold-out room type when no inventory is added) and $F(x_i, D_i = 1) + \sum_{x_j \in S_i} F(x_j, D_i = 1) \geq F(x_i, D_i = 1)$, it is straightforward to prove that:

$$ITE_{\text{platform}}(x_i) \geq ITE'_{\text{platform}}(x_i) = F(x_i, D_i = 1) - \sum_{x_j \in S_i} F(x_j, D_i = 0) \quad (23)$$

Therefore, $ITE'_{\text{platform}}(x_i) \geq 0$ is a sufficient condition for ensuring a positive platform uplift. The inventory addition decision is thus made by evaluating the simplified platform-level ITE:

$$\text{Add inventory if } ITE'_{\text{platform}}(x_i) \geq 0. \quad (24)$$

B Description of Baselines

As there is no prior work on the specific task of hotel inventory addition, we compare CS²Net against seven state-of-the-art causal effect estimation models, all of which estimate Individual Treatment Effect (ITE) to support decision-making. Given that our problem is similarly framed as causal effect estimation, we believe the comparison is fair. Moreover, in our experiments, we applied the same data preprocessing pipeline for all models to ensure consistency in input features. We also selected the best-performing model for each baseline to guarantee optimal results.

- **TARNet** [19]: Introduces the concept of ITE and proposes representation learning strategies to address sample imbalance between the treatment and control groups.
- **Dragonnet** [21]: Incorporates regularization, propensity scoring, and a three-headed neural network to eliminate confounding variables that affect only the outcome.
- **DR-CFR** [10]: Employs three separate representation networks to disentangle covariates affecting treatment assignment and those affecting outcomes, thereby reducing selection bias.
- **EFIN** [15]: Introduces an explicit feature interaction-aware uplift network, capturing treatment-sensitive features and improving causal effect estimation.
- **GANITE** [31]: Proposes a Generative Adversarial Network (GAN)-based approach for inferring ITE, modeling counterfactual distributions through adversarial training.
- **DESCN** [37]: Learns comprehensive treatment, factual, and confounder representations using a cross-network structure, mitigating selection bias through joint representation learning across the entire treatment space.
- **ECUP** [12]: Introduces the Entire Chain Uplift method with context-enhanced learning, devised to tackle chain-bias problem and the treatment-unadaptive problem.

C Detailed Evaluation Metrics

We use three evaluation metrics to assess the effectiveness of inventory addition decisions. AUUC is a widely used metric in uplift modeling that quantifies the cumulative benefit of targeting decisions. Given a ranking $\pi(\rho)$ of samples sorted by predicted Individual Treatment Effect (ITE), AUUC is computed as:

$$AUUC_{\pi(\rho)} = \left(\frac{R_{\pi(\rho)}^{d=1}}{N_{\pi(\rho)}^{d=1}} - \frac{R_{\pi(\rho)}^{d=0}}{N_{\pi(\rho)}^{d=0}} \right) (N_{\pi(\rho)}^{d=1} + N_{\pi(\rho)}^{d=0}) \quad (25)$$

where $\pi(\rho)$ denotes the top ρ proportion of samples when sorted in descending order by the model's predicted ITE score with $\rho \in [0, 1]$. $R_{\pi(\rho)}^{d=1}$, $R_{\pi(\rho)}^{d=0}$ are the number of successful bookings (i.e., positive samples) in the treatment and control groups. $N_{\pi(\rho)}^{d=1}$, $N_{\pi(\rho)}^{d=0}$ are the total number of samples in the treatment and control groups. A higher AUUC indicates better causal ranking and uplift decision-making.

To specifically evaluate the model's performance within the treatment group, we also use two standard ranking metrics. Let U_T be the set of all samples in the treatment group, L_k be the subset containing the top k percent of samples from U_T when ranked in descending order by the model's prediction score. Let $y \in \{0, 1\}$ be the binary indicator for a successful booking for sample i . HiteRate@ k calculates the proportion of all successful bookings that are captured within the top $k\%$ of inventory additions:

$$\text{HitRate@}k = \frac{\sum_{i \in L_k} y_i}{\sum_{i \in U_T} y_i} \quad (26)$$

Precision@ k computes the proportion of successful bookings among the top $k\%$ percent of inventory additions:

$$\text{Precision@}k = \frac{\sum_{i \in L_k} y_i}{|L_k|} \quad (27)$$

These metrics complement each other by evaluating both uplift ranking quality (AUUC) and actionable impact in top-ranked predictions (HitRate, Precision).

D Time Complexity Analysis

Table 5 shows the training and inference costs. While CS²Net's training time is slightly longer than the baselines, its inference speed remains highly competitive. The marginal increase in inference time (1-2 ms over the next slowest models) is negligible in a production setting. Given that real-time performance is paramount for our application, the minor offline training overhead is a worthwhile trade-off for the significant gains in predictive accuracy. This confirms that CS²Net offers a practical and scalable solution that balances state-of-the-art performance with real-world efficiency requirements.

E Sensitivity of Parameters

To systematically evaluate the impact of each hyper-parameter in the loss function (Eq. 19), we tune one hyper-parameter at a time while keeping others fixed at their optimal values from prior experiments. The hyper-parameters are optimized in the order of α_1 , α_2 , α_3 , and α_4 . As summarized in Table 6, the model achieves the best performance with the following settings: $\alpha_1 = 10^{-5}$, $\alpha_2 = 10^{-1}$, $\alpha_3 = 10^{-2}$ and $\alpha_4 = 10^{-4}$. These results indicate

Table 5: Time Complexity Analysis of Different Methods.

Method	Training Time (h)	Inference Time (ms/batch)
TARNet	0.517	950
GANITE	0.567	1050
Dragonnet	0.517	970
DR-CFR	0.550	1030
DESCN	0.567	1052
EFIN	0.567	1053
ECUP	0.566	1052
CS ² Net	0.583	1054

Table 6: Hyperparameter Sensitivity Analysis of CS²Net.

α_1	AUUC	α_2	AUUC	α_3	AUUC	α_4	AUUC
1e-6	0.8188	1e-5	0.8179	1e-5	0.8231	1e-5	0.8309
3e-6	0.8203	1e-4	0.8214	1e-4	0.8300	3e-5	0.8350
5e-6	0.8262	1e-3	0.8391	5e-4	0.8261	5e-5	0.8488
1e-5	0.8400	1e-2	0.8310	1e-3	0.8339	1e-4	0.8587
1e-4	0.8193	5e-2	0.8400	5e-3	0.8419	5e-4	0.8442
5e-4	0.8134	1e-1	0.8479	1e-2	0.8587	1e-3	0.8468
1e-3	0.8147	2e-1	0.8241	5e-2	0.8538	5e-3	0.8451
1e-2	0.8104	3e-1	0.8294	1e-1	0.8479	1e-2	0.8459
1e-1	0.8097	4e-1	0.8264	2e-1	0.8411	1.5e-2	0.8482
2e-1	0.8094	5e-1	0.8232	3e-1	0.8408	2e-2	0.8446

that the adversarial loss requires a relatively small weight to prevent over-regularization, while the disentanglement loss benefits from a higher weighting to effectively separate treatment, confounder, and outcome representations. The contrastive loss and IPM loss exhibit moderate sensitivity, with optimal performance occurring at 10^{-4} and 10^{-2} .

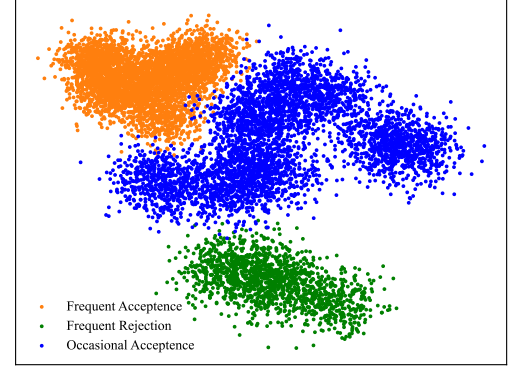
For other parameters, all network parameters are initialized with *Xavier Initialization* [6], where each parameter is sampled from $N(0, \mu^2)$ with $\mu = -\sqrt{2/(n_{in} + n_{out})}$. n_{in} , n_{out} denote the number of input and output neurons, respectively. The input feature dimension is set to 8, and the fully connected layers in the MTP module have dimensions 256, 128, and 1, respectively. For training, we use the mini-batch size of 512 and *Adam* optimizer with a learning rate of 0.001 for training all models. The value of each experimental result is the average of 5 repeated tests.

F Effectiveness of Learned Representations

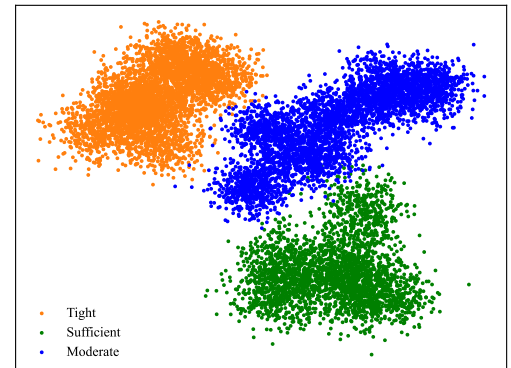
To better understand the semantic quality of the representations learned by our modules, we conduct a visualization-based analysis of the embeddings generated by the Hotel Service Representation (HSR) and External Inventory-Based Scarcity Representation (ERS) modules.

For the hotel service representations, we randomly sample 10,000 hotels on the platform and project their embeddings into a two-dimensional space using t-SNE [25]. We label each hotel based on its real-world reservation acceptance behavior—*frequent acceptance*, *occasional acceptance*, and *frequent rejection*. As shown in Fig. 7, hotels with similar acceptance behavior form well-separated

clusters in the embedding space, indicating that the HSR module effectively captures meaningful patterns related to hotel engagement and responsiveness. Similarly, for 10,000 randomly selected

**Figure 7: Visualization of Hotel Embedding Learned by HSR.**

room types, we visualize their embeddings learned by ERS model and label their actual inventory status—*tight*, *moderate*, and *sufficient*. As illustrated in Fig. 8, room types with similar scarcity levels are closely clustered, suggesting that the ERS module successfully learns inventory scarcity representations that align with real-world availability patterns.

**Figure 8: Visualization of Room Type Embeddings Learned by ERS Module.**

G Sales Shifts Analysis in Online Test

In online test, we also conduct a randomized controlled trials (RCT) to assess platform-level effects and potential sales shifts. In this setting, the Test group served as the Treatment group (using CS²Net),

Table 7: Sales Shift Analysis in Online Test.

Metric	Target Hotels	Similar Hotels
R_s^T (Treatment: CS ² Net)	+1.590%	+0.447%
R_s^C (Control: New Hotels)	+0.468%	+0.453%
$R_d = R_s^T - R_s^C$	+1.122%	−0.006%

while newly onboarded hotels without inventory adjustments formed the Control group.

As shown in Table 7, CS²Net improved overall bookings with minimal sales shifts: the uplift for target hotels reached +1.122%, while similar hotels experienced a negligible decline (−0.006%). This again confirms that CS²Net effectively grows the overall platform revenue with minimal sales displacement.